

Lab 10: Distributed Machine Learning with Apache Spark MLlib

Z5008 Big Data Lab — MTech Data Science & AI

Dr. Innocent Nyalala
`innocent@iitmz.ac.in`

School of Engineering and Science
IIT Madras Zanzibar

Monday, 18 May 2026

Session Outline

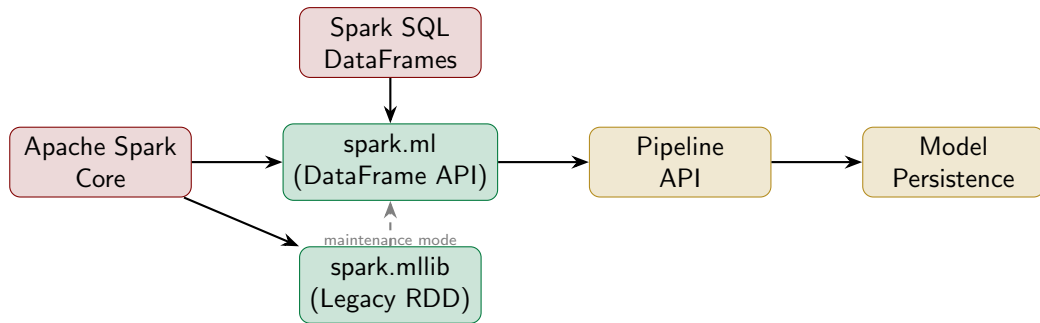
Theory (Frames 3–14)

- 1 MLib Architecture
- 2 RDD vs DataFrame API
- 3 Transformers & Estimators
- 4 Pipelines
- 5 Feature Engineering I (categorical)
- 6 Feature Engineering II (scaling, imputation)
- 7 Feature Engineering III (text, SQL)
- 8 Classification Algorithms I
- 9 Classification Algorithms II
- 10 Classification Algorithms III
- 11 Regression Algorithms

Hands-on (Frames 15–28)

- 15 Building a Full Pipeline
- 16 Confusion Matrix & ROC
- 17 CrossValidator Tuning
- 18 TrainValidationSplit
- 19 Imbalanced Data Strategies
- 20 Model Persistence (S3A)
- 21 Feature Importance
- 22 MLib vs scikit-learn vs XGBoost
- 23 XGBoost4J-Spark
- 24 Distributed Training Strategies
- 25 Lab Exercise Overview
- 26 Common Errors & Debugging

MLlib Architecture Overview



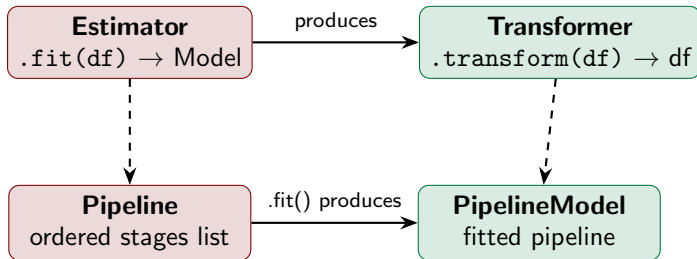
Two API layers

- **spark.mllib** — RDD-based (legacy, maintenance only)
- **spark.ml** — DataFrame-based (**always use this**)

Golden rule

Always import from `pyspark.ml.*`
Never from `pyspark.mllib.*` for new code.

Core Abstractions: Estimator, Transformer, Pipeline, PipelineModel



Estimator examples

LogisticRegression,
RandomForestClassifier,
StandardScaler, StringIndexer, KMeans

Transformer examples

LogisticRegressionModel,
StandardScalerModel,
fitted StringIndexerModel

Feature Engineering I: Categorical Encoding

```
1 from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
2
3 # StringIndexer: maps string labels to indices (0, 1, 2, ...)
4 indexer = StringIndexer(
5     inputCols=["payment_type", "telecom_provider", "region"],
6     outputCols=["payment_type_idx", "provider_idx", "region_idx"],
7     handleInvalid="keep"    # options: error | skip | keep
8 )
9
10 # OneHotEncoder: converts indices to sparse binary vectors
11 encoder = OneHotEncoder(
12     inputCols=["payment_type_idx", "provider_idx", "region_idx"],
13     outputCols=["payment_type_ohe", "provider_ohe", "region_ohe"],
14     dropLast=True          # avoids perfect multicollinearity
15 )
16
17 # VectorAssembler: merges feature columns into one vector
18 assembler = VectorAssembler(
19     inputCols=[
20         "payment_type_ohe", "provider_ohe", "region_ohe",
21         "amount", "balance_before", "balance_after", "customer_age"
22     ],
```

Feature Engineering II: Scaling, Imputation, Discretisation

```
1 from pyspark.ml.feature import (  
2     StandardScaler, MinMaxScaler, Normalizer,  
3     Bucketizer, QuantileDiscretizer, Binarizer,  
4     Imputer, PCA  
5 )  
6  
7 # Imputer: replace missing numerics before assembling  
8 imputer = Imputer(  
9     inputCols=["amount", "balance_before"],  
10    outputCols=["amount_imp", "bal_before_imp"],  
11    strategy="median"           # or "mean"  
12 )  
13  
14 # StandardScaler: zero-mean, unit-variance scaling  
15 scaler = StandardScaler(  
16     inputCol="raw_features", outputCol="features",  
17     withMean=True, withStd=True  
18 )  
19  
20 # QuantileDiscretizer: bin continuous into discrete buckets  
21 disc = QuantileDiscretizer(  
22     numBuckets=5, inputCol="amount", outputCol="amount_bucket"
```

Feature Engineering III: Text & SQL Transformers

```
1 from pyspark.ml.feature import (  
2     Tokenizer, HashingTF, IDF, Word2Vec, SQLTransformer  
3 )  
4  
5 # TF-IDF pipeline for transaction notes / SMS content  
6 tokenizer = Tokenizer(inputCol="sms_text", outputCol="words")  
7 hashing_tf = HashingTF(inputCol="words", outputCol="raw_tf",  
8     numFeatures=20000)  
9 idf = IDF(inputCol="raw_tf", outputCol="tfidf_features",  
10     minDocFreq=5)  
11  
12 # Word2Vec: dense embeddings (100-dim)  
13 w2v = Word2Vec(vectorSize=100, minCount=3, seed=42,  
14     inputCol="words", outputCol="w2v_features")  
15  
16 # SQLTransformer: derive new columns with SQL expressions  
17 sql_tx = SQLTransformer(  
18     statement=""  
19     SELECT *,  
20         (balance_after - balance_before)           AS balance_delta,  
21         (amount / (balance_before + 1e-6))          AS amount_ratio,  
22         CASE WHEN amount > 50000 THEN 1 ELSE 0 END AS large_txn
```

Classification I: Logistic Regression & Decision Tree

```
1 from pyspark.ml.classification import (  
2     LogisticRegression, DecisionTreeClassifier  
3 )  
4  
5 # LogisticRegression: binary and multinomial  
6 lr = LogisticRegression(  
7     featuresCol="features", labelCol="label",  
8     maxIter=100,  
9     regParam=0.01,          # L2 lambda (Ridge component)  
10    elasticNetParam=0.0,     # 0=pure Ridge, 1=pure Lasso  
11    family="binomial",       # "multinomial" or "auto"  
12    threshold=0.5,  
13    standardization=True  
14 )  
15  
16 # DecisionTreeClassifier  
17 dt = DecisionTreeClassifier(  
18     featuresCol="features", labelCol="label",  
19     maxDepth=10,  
20     maxBins=64,  
21     minInstancesPerNode=5,  
22     impurity="gini",        # or "entropy"
```


Classification II: Random Forest & GBT

```
1 from pyspark.ml.classification import (  
2     RandomForestClassifier, GBTClassifier  
3 )  
4  
5 # RandomForestClassifier: bagging ensemble of decision trees  
6 rf = RandomForestClassifier(  
7     featuresCol="features", labelCol="label",  
8     numTrees=100,  
9     maxDepth=10,  
10    featureSubsetStrategy="auto", # "sqrt"/"log2"/"fraction"/"all"  
11    subsamplingRate=0.8,          # row subsampling per tree  
12    seed=42  
13 )  
14  
15 # GBTClassifier: sequential boosting (binary classification only)  
16 gbt = GBTClassifier(  
17     featuresCol="features", labelCol="label",  
18     maxIter=100,             # number of boosting rounds  
19     stepSize=0.1,           # learning / shrinkage rate  
20     maxDepth=5,  
21     subsamplingRate=0.8,  
22     lossType="logistic",
```

Classification III: LinearSVC, NaiveBayes, MLP

```
1 from pyspark.ml.classification import (  
2     LinearSVC, NaiveBayes, MultilayerPerceptronClassifier  
3 )  
4  
5 # LinearSVC: max-margin binary classifier  
6 svc = LinearSVC(  
7     featuresCol="features", labelCol="label",  
8     maxIter=100, regParam=0.01, tol=1e-6  
9 )  
10  
11 # NaiveBayes: suited to non-negative sparse features (TF-IDF)  
12 nb = NaiveBayes(  
13     featuresCol="tfidf_features", labelCol="label",  
14     modelType="multinomial", # "bernoulli" / "gaussian"  
15     smoothing=1.0  
16 )  
17  
18 # MultilayerPerceptronClassifier: feedforward neural network  
19 # layers = [input_dim, hidden1, hidden2, ..., num_classes]  
20 mlp = MultilayerPerceptronClassifier(  
21     featuresCol="features", labelCol="label",  
22     layers=[50, 64, 32, 2], # 50 features, 2 classes
```

Regression Algorithms

```
1 from pyspark.ml.regression import (  
2     LinearRegression, DecisionTreeRegressor,  
3     RandomForestRegressor, GBTRegressor,  
4     GeneralizedLinearRegression  
5 )  
6  
7 # LinearRegression with ElasticNet  
8 lr_r = LinearRegression(featuresCol="features", labelCol="txn_amount",  
9                           maxIter=100, regParam=0.01, elasticNetParam=0.5)  
10  
11 # Random Forest Regressor  
12 rf_r = RandomForestRegressor(featuresCol="features",  
13                              labelCol="txn_amount",  
14                              numTrees=100, maxDepth=8, seed=42)  
15  
16 # GBT Regressor  
17 gbt_r = GBTRegressor(featuresCol="features", labelCol="txn_amount",  
18                       maxIter=100, stepSize=0.05, maxDepth=6)  
19  
20 # GeneralizedLinearRegression: supports multiple response families  
21 glm = GeneralizedLinearRegression(  
22     featuresCol="features", labelCol="txn_count",
```

Clustering Algorithms

```
1 from pyspark.ml.clustering import (  
2     KMeans, BisectingKMeans, GaussianMixture, LDA  
3 )  
4  
5 # KMeans: centroid-based hard clustering  
6 km = KMeans(featuresCol="features", k=5,  
7             initMode="k-means++", # scalable k-means++ variant  
8             initSteps=5, maxIter=20, seed=42)  
9 km_model = km.fit(df)  
10 print("WSSSE:", km_model.summary.trainingCost)  
11  
12 # BisectingKMeans: divisive hierarchical variant  
13 bkm = BisectingKMeans(featuresCol="features", k=5,  
14                       maxIter=20, minDivisibleClusterSize=1.0)  
15  
16 # GaussianMixture: soft-assignment probabilistic clustering  
17 gmm = GaussianMixture(featuresCol="features", k=5,  
18                       maxIter=100, tol=0.01, seed=42)  
19 gmm_model = gmm.fit(df)  
20 # gmm_model.gaussiansDF -> Gaussian parameters (means, covariances)  
21  
22 # LDA: topic modelling on document-term matrices
```

Collaborative Filtering: ALS

```
1 from pyspark.ml.recommendation import ALS
2 from pyspark.ml.evaluation import RegressionEvaluator
3
4 als = ALS(
5     userCol="customer_id",
6     itemCol="merchant_id",
7     ratingCol="txn_count",          # explicit rating
8     rank=20,                       # number of latent factors
9     maxIter=10,
10    regParam=0.1,
11    implicitPrefs=False,            # True for implicit (view/click) data
12    coldStartStrategy="drop",       # "drop" | "nan"
13    nonnegative=False,
14    seed=42
15 )
16
17 als_model = als.fit(train_df)
18
19 # Top-10 merchant recommendations for every customer
20 user_recs = als_model.recommendForAllUsers(10)
21
22 # Top-5 customers for every merchant
```

Evaluation Metrics

```
1 from pyspark.ml.evaluation import (  
2     BinaryClassificationEvaluator,  
3     MulticlassClassificationEvaluator,  
4     RegressionEvaluator, ClusteringEvaluator  
5 )  
6  
7 # Binary classification  
8 bin_ev = BinaryClassificationEvaluator(labelCol="label",  
9                                     rawPredictionCol="rawPrediction")  
10 auc_roc = bin_ev.evaluate(preds, {bin_ev.metricName: "areaUnderROC"})  
11 auc_pr = bin_ev.evaluate(preds, {bin_ev.metricName: "areaUnderPR"})  
12  
13 # Multiclass (also works for binary)  
14 mc_ev = MulticlassClassificationEvaluator(  
15     labelCol="label", predictionCol="prediction")  
16 for metric in ["accuracy", "f1", "weightedPrecision", "weightedRecall"]:  
17     print(f"{metric}: {mc_ev.evaluate(preds, {mc_ev.metricName: metric}):.4f}")  
18  
19 # Regression  
20 reg_ev = RegressionEvaluator(labelCol="txn_amount",  
21                             predictionCol="prediction")  
22 for m in ["rmse", "mae", "r2"]:
```

Building a Full ML Pipeline

```
1 from pyspark.ml import Pipeline
2 from pyspark.ml.feature import (StringIndexer, OneHotEncoder,
3     VectorAssembler, StandardScaler)
4 from pyspark.ml.classification import RandomForestClassifier
5
6 # 1. Encode target
7 label_idx = StringIndexer(inputCol="status", outputCol="label",
8     handleInvalid="keep")
9
10 # 2. Encode categorical predictors
11 cat_idx = StringIndexer(inputCols=["type", "provider", "region"],
12     outputCols=["type_idx", "prov_idx", "rgn_idx"])
13 cat_ohe = OneHotEncoder(inputCols=["type_idx", "prov_idx", "rgn_idx"],
14     outputCols=["type_ohe", "prov_ohe", "rgn_ohe"])
15
16 # 3. Assemble feature vector
17 asm = VectorAssembler(
18     inputCols=["type_ohe", "prov_ohe", "rgn_ohe",
19         "amount", "balance_before", "customer_age"],
20     outputCol="raw_features", handleInvalid="skip")
21
22 # 4. Scale
23 scaler = StandardScaler(inputCol="raw_features",
24     outputCol="features",
25     withMean=True, withStd=True)
```

Confusion Matrix and ROC Curve

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from pyspark.ml.functions import vector_to_array
4 import pyspark.sql.functions as F
5
6 # Confusion matrix from predictions DataFrame
7 cm = (preds.groupBy("label", "prediction")
8         .count()
9         .toPandas()
10        .pivot(index="label", columns="prediction", values="count"))
11 print(cm)
12
13 # ROC curve: collect probabilities to driver
14 preds_pd = (preds
15             .select("label", vector_to_array("probability").alias("prob"))
16             .withColumn("prob_pos", F.col("prob")[1])
17             .toPandas())
18
19 from sklearn.metrics import roc_curve, auc as sk_auc
20 fpr, tpr, _ = roc_curve(preds_pd["label"], preds_pd["prob_pos"])
21 roc_auc = sk_auc(fpr, tpr)
22
```


Hyperparameter Tuning: ParamGridBuilder + CrossValidator

```
1 from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
2 from pyspark.ml.evaluation import BinaryClassificationEvaluator
3
4 evaluator = BinaryClassificationEvaluator(labelCol="label",
5                                         metricName="areaUnderROC")
6
7 # Build grid: 3 x 3 x 2 = 18 combinations
8 param_grid = (
9     ParamGridBuilder()
10     .addGrid(rf.numTrees, [50, 100, 200])
11     .addGrid(rf.maxDepth, [5, 10, 15])
12     .addGrid(rf.featureSubsetStrategy, ["sqrt", "log2"])
13     .build()
14 )
15 # 18 combinations x 5 folds = 90 Spark training jobs
16
17 cv = CrossValidator(
18     estimator=pipeline, # wrap the full Pipeline
19     estimatorParamMaps=param_grid,
20     evaluator=evaluator,
21     numFolds=5,
22     seed=42,
```

TrainValidationSplit: Faster Alternative

```
1 from pyspark.ml.tuning import TrainValidationSplit
2
3 tvs = TrainValidationSplit(
4     estimator=pipeline,
5     estimatorParamMaps=param_grid,
6     evaluator=evaluator,
7     trainRatio=0.8,          # 80% train / 20% validation (single split)
8     seed=42,
9     parallelism=4
10 )
11 tvs_model = tvs.fit(train_df)
12 best_tvs = tvs_model.bestModel
13
14 for metric, params in zip(tvs_model.validationMetrics, param_grid):
15     print(f"AUC={metric:.4f}    params={params}")
```

CrossValidator

- *k*-fold cross-validation

TrainValidationSplit

- Single train/val split

Imbalanced Data: Class Weights & Sampling

```
1 import pyspark.sql.functions as F
2
3 total = df.count()
4 n_pos = df.filter(F.col("label") == 1).count()
5 n_neg = total - n_pos
6 ratio_neg = n_neg / total
7 ratio_pos = n_pos / total
8
9 # Strategy 1: class weights in LogisticRegression / LinearSVC
10 df_w = df.withColumn(
11     "class_weight",
12     F.when(F.col("label") == 1, 1.0 / (2 * ratio_pos))
13     .otherwise(1.0 / (2 * ratio_neg))
14 )
15 lr_w = LogisticRegression(weightCol="class_weight",
16                             featuresCol="features", labelCol="label")
17
18 # Strategy 2: undersampling majority class
19 neg_sample = df.filter(F.col("label") == 0).sample(
20     n_pos / n_neg, seed=42)
21 balanced = df.filter(F.col("label") == 1).union(neg_sample)
22
```

Model Persistence: Save & Load via S3A (MinIO)

```
1 from pyspark.sql import SparkSession
2
3 spark = (SparkSession.builder
4         .master("spark://spark-master:7077")
5         .appName("Lab10_MLlib")
6         .config("spark.hadoop.fs.s3a.endpoint",      "http://minio:9000")
7         .config("spark.hadoop.fs.s3a.access.key",    "minioadmin")
8         .config("spark.hadoop.fs.s3a.secret.key",    "minioadmin")
9         .config("spark.hadoop.fs.s3a.path.style.access", "true")
10        .config("spark.hadoop.fs.s3a.impl",
11                "org.apache.hadoop.fs.s3a.S3AFileSystem")
12        .getOrCreate())
13
14 MODEL_PATH = "s3a://z5008/models/lab10_rf_pipeline"
15
16 # --- Save fitted PipelineModel ---
17 pm.write().overwrite().save(MODEL_PATH)
18 print(f"Model saved to {MODEL_PATH}")
19
20 # --- Load back ---
21 from pyspark.ml import PipelineModel
22 loaded_model = PipelineModel.load(MODEL_PATH)
```

Feature Importance from Tree-Based Models

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Extract the RF stage from the fitted PipelineModel
5 rf_model = pm.stages[-1] # RandomForestClassificationModel
6 feature_names = asm.getInputCols() # VectorAssembler column list
7
8 # featureImportances is a SparseVector; convert to dense numpy array
9 importances = rf_model.featureImportances.toArray()
10
11 fi = (pd.DataFrame({"feature": feature_names,
12                    "importance": importances})
13      .sort_values("importance", ascending=False)
14      .reset_index(drop=True))
15 print(fi.head(10).to_string(index=False))
16
17 # Horizontal bar chart (top 15)
18 top15 = fi.head(15)
19 plt.figure(figsize=(8, 5))
20 plt.barh(top15["feature"][::-1], top15["importance"][::-1],
21          color="steelblue", edgecolor="white")
22 plt.xlabel("Importance Score (Gini impurity reduction)")
```

MLlib vs scikit-learn vs XGBoost4J-Spark

Criterion	scikit-learn	Spark MLlib	XGBoost4J
Data scale	$\leq$ RAM (1 node)	TBs (distributed)	TBs (distributed)
Algorithm breadth	Very wide	Good	Gradient boosting
Training speed	Fastest (small data)	Good (large data)	Very fast (optimised)
Hyperparameter tuning	GridSearchCV	CrossValidator	XGBoostCV
GPU support	Limited	Experimental	Yes
Integration	Standalone	Spark ecosystem	Spark ecosystem
Deployment	Pickle / ONNX	MLflow / Spark	MLflow / Spark

Use scikit-learn

Data fits in RAM; rapid prototyping; widest algorithm

Use Spark MLlib

>10 GB data; need distributed feature engineering

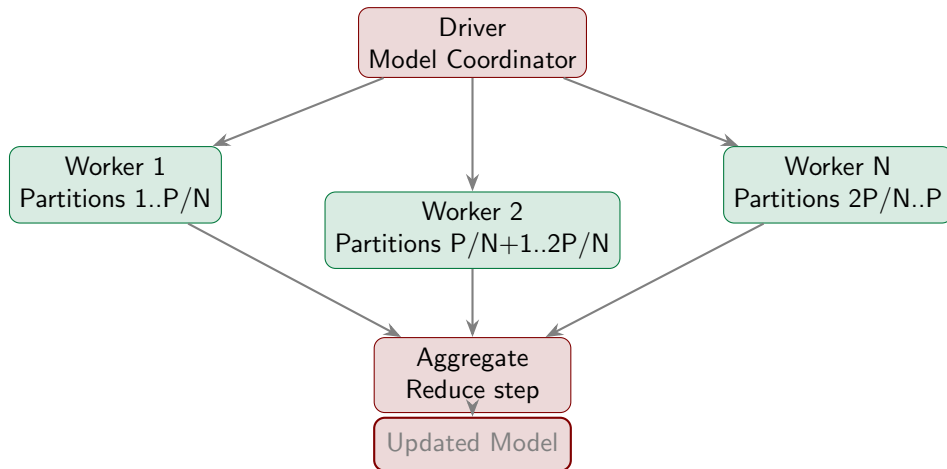
Use XGBoost4J

Need best gradient-boosting accuracy at TB scale; GPU

XGBoost4J-Spark: Drop-in Classifier

```
1 # Install: pip install xgboost (includes Spark bindings)
2 from xgboost.spark import SparkXGBClassifier
3
4 xgb = SparkXGBClassifier(
5     features_col="features",
6     label_col="label",
7     num_boost_round=200,
8     max_depth=6,
9     eta=0.1,                # learning rate (shrinkage)
10    subsample=0.8,
11    colsample_bytree=0.8,
12    objective="binary:logistic",
13    eval_metric="auc",
14    use_gpu=False,          # True if GPU executor nodes available
15    num_workers=4           # should match Spark executor count
16 )
17
18 # Plugs directly into a Spark Pipeline -- same interface as MLlib
19 pipeline_xgb = Pipeline(stages=[label_idx, cat_idx, cat_ohe,
20                                asm, scaler, xgb])
21 xgb_model = pipeline_xgb.fit(train_df)
22 xgb_preds = xgb_model.transform(test_df)
```

Distributed Training Strategies in Spark MLlib



Data Parallelism (Spark default)

Ensemble Parallelism

Lab Notebook Exercise — Step by Step

- ➊ **Setup** — SparkSession + S3A config; load `mobile_money.csv` from MinIO
- ➋ **EDA** — schema, value counts, missing values, class imbalance
- ➌ **Feature engineering** — Imputer, SQLTransformer, StringIndexer, OHE, VectorAssembler, StandardScaler
- ➍ **Logistic Regression baseline** — full Pipeline; accuracy, F1, AUC-ROC
- ➎ **Random Forest** — fit, evaluate, feature importance plot
- ➏ **GBT Classifier** — fit, compare metrics in table
- ➐ **CrossValidator** — ParamGridBuilder over `numTrees` and `maxDepth`; best params
- ➑ **Imbalanced data** — class weights on LR; compare vs. baseline
- ➒ **ALS** — customer-merchant recommendation; RMSE
- ➓ **KMeans clustering** — segment customers; silhouette score
- ➔ **Model persistence** — save best model to `s3a://z5008/models/`; load & verify

Common Errors & Debugging Tips

Common errors

- Column features must be VectorUDT — VectorAssembler missing or wrong outputCol
- handleInvalid=error: unseen label — set handleInvalid="keep" on StringIndexer
- NaN in ALS predictions — set coldStartStrategy="drop"
- NaiveBayes requires non-negative features — apply MinMaxScaler before NB
- GBT only supports binary classification — use RF for multiclass
- CrossValidator OOM — reduce parallelism or reduce grid size

Useful debug commands

```
# Spark UI (DAG, stages, tasks)
open http://localhost:4040

# Check executor memory config
spark.conf.get("spark.executor.memory")

# Inspect Pipeline stages
for i, s in enumerate(model.stages):
    print(i, type(s).__name__)

# DataFrame schema & nulls
df.printSchema()
df.select([F.count(F.when(
    F.col(c).isNull(), c))
    .alias(c) for c in df.columns]
).show()
```

Data Leakage and How MLlib Pipelines Prevent It

What is data leakage?

Leakage occurs when information *from outside the training set* influences model training, producing unrealistically optimistic evaluation scores.

Leaky workflow (WRONG)

- 1 Fit StandardScaler on **all** data
- 2 Split into train / test
- 3 Train model on scaled train set
- 4 Evaluate on scaled test set

Test-set statistics influenced the scaler — **leakage**.

Pipeline workflow (CORRECT)

- 1 Split *raw* data into train / test
- 2 `pipeline.fit(train_df)` — scaler & indexers see train only
- 3 `pm.transform(test_df)` — applies learned parameters to test
- 4 Evaluate on transformed test set

Test set truly unseen — **no leakage**.

Key takeaway

Summary & Next Steps

Today we covered

- MLlib core abstractions: Estimator, Transformer, Pipeline, PipelineModel
- 13 feature engineering transformers
- 7 classifiers (LR, DT, RF, GBT, SVC, NB, MLP)
- 5 regression models (LR, DT, RF, GBT, GLM)
- Clustering: KMeans, BisectingKMeans, GMM, LDA
- Collaborative filtering: ALS
- Evaluation: AUC-ROC/PR, F1, RMSE, Silhouette
- CrossValidator vs TrainValidationSplit
- Imbalanced data: weights + sampling

Week 10 Assignment (100 marks)

Deadline: **24 May 2026, 23:59 EAT**

- Task 1 (15): EDA + feature engineering
- Task 2 (25): Full Pipeline + evaluation
- Task 3 (25): CrossValidator tuning
- Task 4 (20): Second model + feature importance
- Task 5 (15): Conceptual reflection

Lab 11 (next week)